

A Robot-Action-Planning FPGA Accelerator With Flow-Predictive Inference and Adaptive Real-Time Chunking for Embodied AI

Haozhe Zhu¹, *Member, IEEE*, Yujie Ma, *Student Member, IEEE*, Siqi He², *Graduate Student Member, IEEE*,
 Linqi Li, *Student Member, IEEE*, Sinong Gong, *Student Member, IEEE*,
 Mengjie Li³, *Graduate Student Member, IEEE*, Xiankui Xiong, Chengchen Wang⁴, Haidong Tian,
 Qi Liu⁵, *Member, IEEE*, Xiaoyang Zeng⁶, *Senior Member, IEEE*, and Chixiao Chen⁷, *Member, IEEE*

Abstract—Vision-language-action (VLA) models have shown strong potential for robotic manipulation, yet deploying their diffusion-based action experts on edge platforms remains challenging due to the high inference latency, the overhead of real-time chunking (RTC), and the large memory footprint of model parameters and intermediate activations. This article presents an FPGA accelerator for real-time action planning in embodied robotic systems. The accelerator integrates three co-designed techniques. First, a velocity-estimation-driven flow predictor exploits the near-linear evolution of flow velocity across diffusion steps in rectified-flow-based inference to improve planning throughput. Second, an adaptive RTC scheduler dynamically switches between gradient-based and rectified guidance according to robotic states, thereby reducing computational overhead and energy consumption during asynchronous planning. Third, a layer-fusion and FFN-partition dataflow minimizes off-chip traffic for intermediate activations and reduces the on-chip memory requirement. Implemented for the action expert of the π_0 VLA model, the proposed design achieves 903.2 GOPS and 111.09 GOPS/W. In asynchronous action planning, it sustains a control rate of 77.48 Hz with an energy efficiency of 85 mJ/token, and is validated in both simulation and real-robot deployment on an SO-101 platform. These results demonstrate that the proposed accelerator enables real-time and energy-efficient edge-side deployment of VLA action planning for embodied robotic control.

Index Terms—Vision-language-action (VLA) model, embodied artificial intelligence, FPGA accelerator, real-time chunking (RTC), diffusion transformer.

Received 25 April 2026; revised 30 July 2026 and 24 August 2026; accepted 1 September 2026. This work was supported in part by the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China under Grant JYB2025XDXM120, in part by the State Key Laboratory of Integrated Chips and Systems (SKLICS) under Grant SKLICS-Z202502, and in part by ZTE Industry-University-Institute Cooperation Funds under Grant IA20241120003-PO0002. This article was recommended by Associate Editor J. Tian. (Haozhe Zhu and Yujie Ma contributed equally to this work.) (Corresponding author: Haozhe Zhu.)

Haozhe Zhu, Yujie Ma, Siqi He, Linqi Li, Sinong Gong, Mengjie Li, Qi Liu, Xiaoyang Zeng, and Chixiao Chen are with the State Key Laboratory of Integrated Chips and Systems (SKLICS), Fudan University, Shanghai 200433, China (e-mail: zhuhz@fudan.edu.cn).

Xiankui Xiong, Chengchen Wang, and Haidong Tian are with the State Key Laboratory of Mobile Network and Mobile Multimedia Technology, ZTE Corporation, Shenzhen 518057, China.

Digital Object Identifier 10.1109/TCSI.2026.3732698

I. INTRODUCTION

EMBODIED artificial intelligence (AI) is driving a growing demand for action-planning engines that are both accurate and responsive [1]. For robotic manipulation, high-quality decisions must be generated under tight control-loop deadlines, making real-time action planning a key requirement for practical deployment. Recent progress in vision-language-action (VLA) models [2], [3], [4], [5], [6], [7] has significantly improved the generality and fidelity of robot policies by jointly exploiting visual observations, language instructions, and action generation.

As depicted in Fig. 1, a typical VLA pipeline consists of two stages. In the front-end stage, a vision-language model (VLM) encodes image and text inputs into multimodal features or a key-value (KV) cache. In the back-end stage, an action expert generates action chunks conditioned on the encoded context. In diffusion-based VLA models such as π_0 [2], the action expert is commonly implemented with a diffusion transformer (DiT) [8], which iteratively refines noisy action latents into executable motor commands. Since the VLM is relatively less latency-sensitive and can be hosted in the cloud [9], the edge-side action expert becomes the critical component for real-time robotic control.

However, efficient edge deployment of the action expert remains difficult for three reasons. First, flow-matching-based DiT inference [10] requires multiple diffusion iterations, resulting in substantial latency and making it difficult to satisfy control rates above 50 Hz. Second, real-time chunking (RTC) [11], which overlaps action generation with robot execution for asynchronous planning, introduces additional synchronization overhead due to gradient-based flow guidance. Third, the action expert must handle large model weights, KV cache, and intermediate activations under a tightly constrained on-chip memory budget, leading to frequent off-chip accesses and degraded energy efficiency. These challenges make conventional edge processors and generic Transformer accelerators insufficient for real-time VLA action planning.

Several recent works have explored hardware support for robotic diffusion inference and edge-side VLA execution. An edge SoC for diffusion-based robotic manipulation [12] mitigates redundant diffusion steps through a

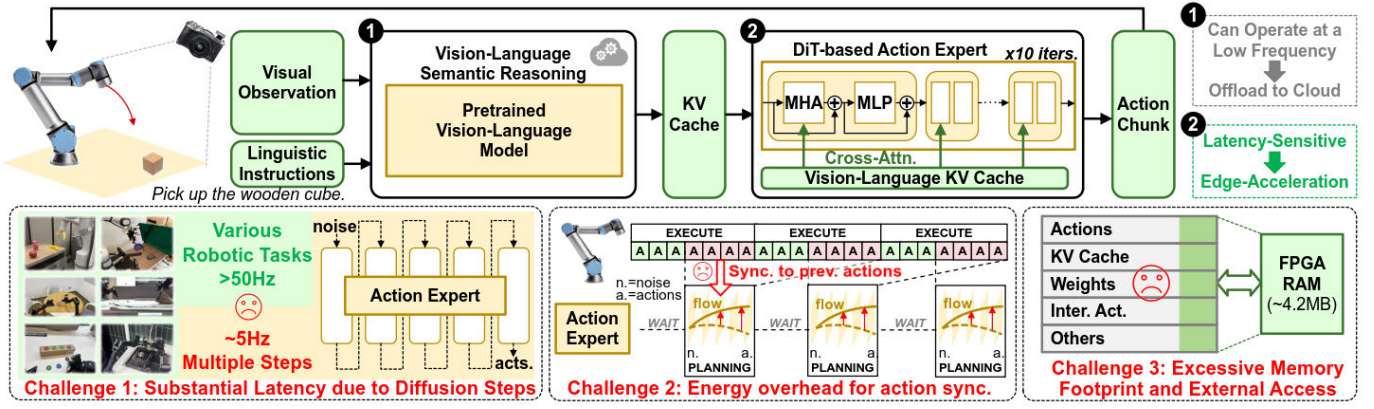


Fig. 1. VLA pipeline and key challenges for edge-side action planning: multi-step diffusion latency, RTC synchronization overhead, and large memory footprint.

Speculation-Commitment-Dispatcher (SCD) based on the iteration increments. ActionFlow [13] proposes an inference framework with cross-request pipelining and KV buffer optimization, tailored for resource-constrained edge platforms. Beyond VLA-specific designs, the action expert in VLA models is fundamentally built upon Transformer-based architectures, making prior accelerator designs for vision and language models [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24] relevant. These works provide efficient support for attention and matrix operations, but are not designed to exploit the iterative structure of flow-matching inference or the mode-dependent computation pattern introduced by RTC, limiting their efficiency for embodied action planning.

To address these issues, this article presents an FPGA accelerator for diffusion-based VLA action experts with flow-prediction inference, adaptive RTC scheduling, and memory-efficient dataflow optimization. The key idea is to co-design the inference algorithm and accelerator architecture around the structured properties of rectified flow and asynchronous chunked execution. Specifically, this work makes the following contributions:

- **Flow-predictive inference:** We propose a velocity-estimation-driven flow predictor that exploits the approximately linear evolution of flow velocity across diffusion steps in rectified-flow-based inference, thereby bypassing redundant diffusion iterations and reducing inference latency with limited accuracy degradation.
- **Adaptive RTC scheduling:** We propose a dual-mode RTC mechanism that dynamically switches between the original gradient-based guidance and the proposed lightweight rectified guidance based on robotic states, reducing energy overhead associated with action alignment during asynchronous planning.
- **Memory-efficient accelerator dataflow:** We propose a layer-fusion and FFN-partition dataflow that reduces off-chip traffic for intermediate activations and lowers the effective on-chip storage requirement for large Transformer layers.
- **FPGA prototype and system-level validation:** We implement the proposed design on a Xilinx XCZU7EV

FPGA and deploy the π_0 action expert end to end. The prototype achieves 903.2 GOPS and 111.09 GOPS/W, sustaining a 77.48-Hz control rate at 85 mJ/token in asynchronous action planning, and is further validated in both MuJoCo simulation [25] and real-robot deployment on an SO-101 platform [26].

The remainder of this article is organized as follows. Section II reviews the background of diffusion-based VLA models, flow matching, and RTC. Section III presents the overall accelerator architecture. Sections IV and V describe the flow predictor and the adaptive RTC scheduler. Section VI introduces the layer-fusion and FFN-partition dataflow. Section VII reports implementation and evaluation results, and Section VIII concludes this article. This article is an extended version of the conference publication presented at [27].

II. BACKGROUND AND MOTIVATION

A. Diffusion-Based VLA Action Expert

DiT-based VLA models generate actions through iterative refinement of latent representations conditioned on multimodal inputs. As defined in Eq. 1, a typical VLA model consists of two main components: (1) The VLM encodes visual tokens $\mathbf{x}_V$ and the linguistic tokens $\mathbf{x}_L$ into a unified representation, stored as a KV cache. (2) The DiT-based action expert then generates the action field $\mathbf{x}_A$ by iteratively refining a latent from an initial Gaussian noise $\mathbf{x}_{\text{noise}}$ over N diffusion steps. During each step, action tokens interact with the visual-linguistic KV cache through cross-attention, and the final action chunk is obtained by sampling from the refined action field.

$$\begin{cases} \mathbf{k}_{VL}, \mathbf{v}_{VL} = \text{VLM}(\mathbf{x}_V, \mathbf{x}_L) \\ \mathbf{x}_A = \text{AE}_{\mathbf{k}_{VL}, \mathbf{v}_{VL}}^{(N)}(\mathbf{x}_{\text{noise}}) \end{cases} \quad (1)$$

The VLM usually contains billions of parameters and processes long token sequences, incurring substantial computational and memory-access overhead. Therefore, it is generally suitable for cloud-side execution when network connectivity is available [9]. In contrast, the DiT-based action expert is more latency-critical because it directly determines the action-generation rate of the robot. Although it is smaller

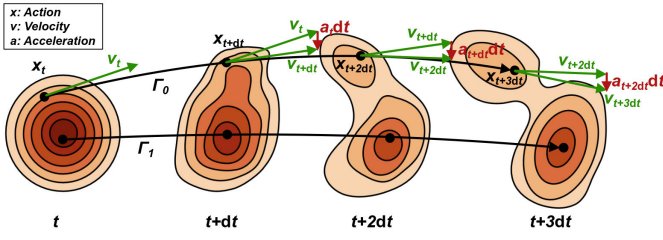


Fig. 2. Action latent evolution under the flow-matching formulation.

than the VLM, it still contains hundreds of millions of parameters and performs repeated attention and FFN operations across diffusion steps. This combination of latency sensitivity, iterative computation, and structured data reuse motivates dedicated edge-side acceleration for the action expert.

B. Flow-Matching-Based Action Generation

Flow matching [10] has emerged as a widely adopted paradigm for the DiT-based action expert in VLA models, as shown in Fig. 2. Unlike conventional denoising formulations that progressively predict and remove noise, the flow-matching strategy is characterized by (Eq. 2): (1) The action expert takes current flow field x_t as an input to infer a velocity v_t . (2) The flow is updated by the velocity over a time step. Throughout the iterative refinement from $t = 0$ to $t = 1$, the flow field is initialized as a standard Gaussian distribution $x_0 \sim \mathcal{N}(0, I)$ and converges to the target action distribution $x_1 \sim \mathcal{N}(\mu, \sigma_{\min})$. The output action chunks are sampled from this final distribution.

$$\begin{cases} v_t = \text{AE}(x_t) \\ x_{t+dt} = x_t + v_t dt \end{cases} \quad (2)$$

In rectified-flow-based training [28], the transformation between the initial noise x_0 and target action x_1 is modelled as a linear trajectory:

$$x_t = (1-t)x_0 + tx_1 \quad \text{i.e.} \quad v_t = \frac{x_1 - x_0}{1-t} \quad (3)$$

This formulation encourages a smooth and nearly straight latent trajectory from noise to action. In practice, such trajectory regularity often leads to approximately linear evolution of the predicted flow velocity v_t across discrete diffusion steps. This observation motivates velocity extrapolation: later diffusion steps can be predicted from early-step velocity statistics, thereby reducing redundant action expert invocations.

C. Real-Time Chunking for Asynchronous Planning

Conventional VLA models follow a synchronous planning-execution loop constrained by observation dependency. As depicted in Fig. 3(a), the processor captures environmental observations only after the preceding action chunk has been executed completely, then initiates the VLA model inference and generates the subsequent action chunk. This approach inherently requires action planning to depend on the most recent environmental state, forcing the robot into idle states during action planning and preventing seamless real-time control.

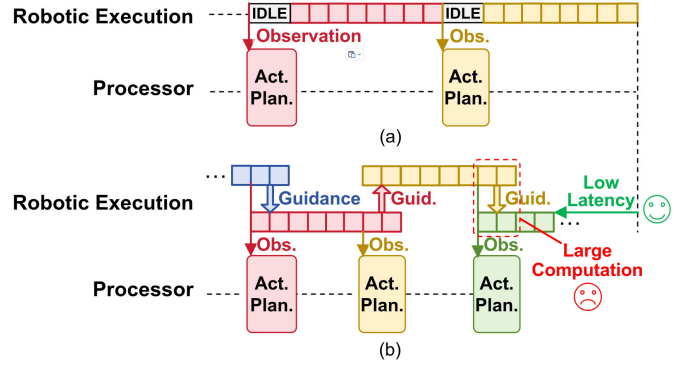


Fig. 3. (a) Synchronous action planning. (b) Asynchronous planning with RTC.

RTC [11] enables asynchronous action planning, mitigating the observation gap through flow guidance during diffusion. As depicted in Fig. 3(b), RTC flow guidance ensures consistency between the initial actions of the subsequent chunk and the final actions of the preceding chunk. Leveraging this continuity between the consecutive chunks, action planning commences with the overlapped actions under execution. The subsequent chunk is executed skipping the initial actions already executed during generation. The RTC strategy enables seamless robotic action execution without compromising action fidelity.

The flow-guidance process is formulated in Eq. 4. Here, x denotes the flow field, v denotes the flow velocity, and a denotes the final action field at the end of the diffusion process. After each action expert inference step, flow guidance corrects the unguided velocity v_t to obtain the guided velocity v_t^* , thereby steering the final action field toward the target field a^* . (1) The action expert computes the unguided flow velocity v_t from the current flow field x_t at diffusion time t . (2) The final action field associated with x_t is represented by $\hat{a}_t$. Since the final field remains unavailable at the intermediate diffusion time, $\hat{a}_t$ is estimated by linearly extrapolating x_t along v_t . (3) For each action index i , the guided velocity $v_t^{*(i)}$ is obtained by adding a correction proportional to the deviation between the target field $a^{*(i)}$ and the estimated final field $\hat{a}_t^{(i)}$, weighted by the local sensitivity $\partial \hat{a}_t^{(i)} / \partial x_t^{(i)}$. The action-wise guidance weight w_i is set to 1 for the overlapped actions and 0 for the remaining actions, thereby restricting flow guidance to the overlapped action fields. The diffusion-dependent coefficient $r_t = \frac{1-t}{t} + \frac{t}{1-t}$ [11] modulates the guidance strength at diffusion time t .

$$\begin{cases} v_t = \text{AE}_t(x_t) \\ \hat{a}_t = \text{AE}_{t,1}(x_t) \approx x_t + (1-t)v_t \\ v_t^{*(i)} = v_t^{(i)} + r_t w_i (a^{*(i)} - \hat{a}_t^{(i)})^T \frac{\partial \hat{a}_t^{(i)}}{\partial x_t^{(i)}} \end{cases} \quad (4)$$

The RTC strategy requires gradient computation for flow guidance, resulting in substantial computational and memory-access overhead. This computational overhead motivates a lightweight strategy that reduces computational complexity while preserving alignment with the overlapped actions.

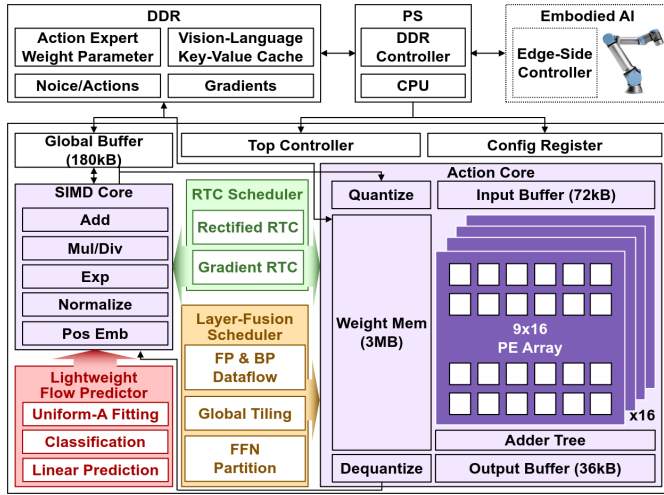


Fig. 4. Top-level architecture of the developed FPGA acceleration system.

III. ACCELERATOR ARCHITECTURE

Fig. 4 illustrates the top-level architecture of the robotic manipulation acceleration system comprising four main components: (1) an embodied robotic device, such as a robotic arm or a dexterous hand, equipped with an edge-side controller; (2) an FPGA accelerator dedicated to the action expert of VLA models; (3) a DDR memory for storing model parameters and activations during action planning; and (4) a CPU serving as the top-level system controller. Before the action expert processing commences, the DDR is preloaded with model parameters and visual-linguistic KV cache. During inference, the intermediate data including noise inputs, action outputs and gradient tensors are maintained in DDR. The CPU orchestrates data transfers and manages computation flow on the accelerator, meanwhile ensuring synchronization between data loading and consumption. Upon the completion of action planning, the CPU commits the action chunks to the edge-side controller for motor command transform and execution.

The FPGA-domain accelerator architecture integrates a comprehensive framework to efficiently process the action expert workloads. The accelerator comprises a global controller, a register block for runtime configuration, a dedicated action core for intensive tensor operations, a SIMD core for miscellaneous vector operations and a global buffer for intermediate data storage. The action core serves as the computation backbone of the accelerator, featuring a processing engine composed of a 16-bank 9×16 INT8 processing element (PE) array for high-throughput matrix multiplication, an adder tree for reduction summation, quantization and dequantization units for seamless conversion between INT8 and FP16 representations. The SIMD core is specifically designed to handle the nonlinear operations prevalent in the action experts, including bypass aggregations, activation functions, positional embeddings and normalizations. These complex operations are decomposed into fundamental arithmetic primitives and executed through a deeply pipelined datapath. Regarding the on-chip memory resources, a 180 KB global buffer stores intermediate activations and residual tokens. The memories

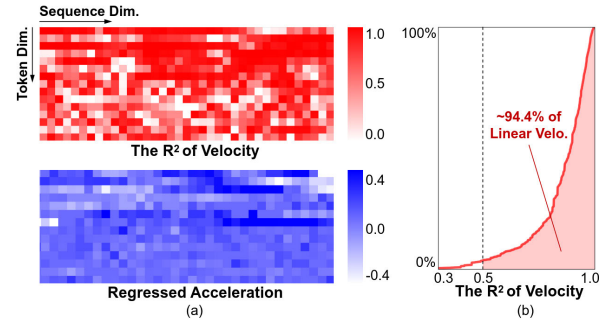


Fig. 5. Statistical characterization of flow velocity: (a) heatmaps of velocity linearity and regressed acceleration; (b) CDF of the velocity R^2 values.

near the action core stage operands and results for the action processing engine, comprising a 3 MB weight memory, a 72 KB input activation buffer, and a 36 KB output activation buffer.

The accelerator incorporates specialized circuits for performance optimization, including a lightweight flow predictor and an adaptive RTC scheduler. During the iterative diffusion, the flow predictor fits the early-step velocity, enabling predictive inference to bypass later-step redundant operations. The adaptive RTC scheduler switches between the original gradient-guidance strategy and the proposed rectified-guidance strategy to reduce energy consumption of the action core. Moreover, the accelerator supports the layer-fusion and FPN-partition dataflow. The hierarchical dataflow enables immediate consumption of the intermediate activations and partitioning of the large layers into smaller sublayers, significantly reducing on-chip memory footprint and alleviating computational stalls.

IV. FLOW-PREDICTIVE INFERENCE

A. Velocity-Estimation-Based Prediction Algorithm

While flow matching preserves action planning quality, its iterative diffusion process incurs substantial inference latency. Rectified-flow-based pre-training models the transformation from initial noise to target actions along an approximately linear trajectory, thereby promoting continuous and smooth evolution of the action field. Consequently, the flow velocity exhibits uniform evolution across diffusion steps.

To characterize velocity evolution across diffusion steps, we randomly sampled 200 robot trajectories, comprising 25.3k actions, from the LIBERO-Spatial and LIBERO-Object task suites. Each action chunk contains 50 actions, with each action represented by a 32-dimensional vector. An action is classified as exhibiting linear velocity evolution when more than half of its velocity dimensions achieve an R^2 value greater than 0.5. As shown in Fig. 5, statistical analysis of π_0 reveals that approximately 94.4% of the sampled actions satisfy this criterion across diffusion steps. This regularity reveals substantial redundancy in velocity computation and motivates skipping diffusion steps to reduce inference latency.

Based on this observation, a velocity-estimation-driven flow prediction algorithm is developed to extrapolate later-step velocities from early-step statistics, thereby bypassing

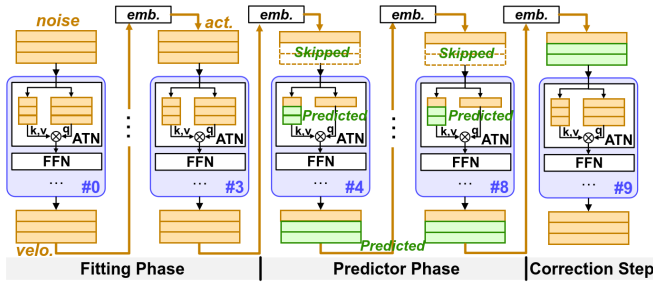


Fig. 6. The three-phase strategy of the flow prediction.

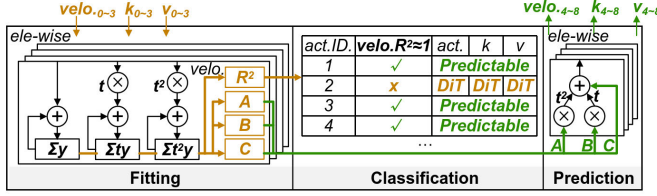


Fig. 7. The circuit implementation of the flow predictor.

redundant action expert processing and reducing inference latency. As depicted in Fig. 6, the N -step diffusion process is partitioned into three distinct stages: (1) A fitting phase comprises the early $(N/2 - 1)$ steps, where the predictor computes the coefficient of determination (R^2) of velocities across these steps. Actions exhibiting velocity linearity patterns are identified as predictable via threshold-based R^2 discrimination. (2) A prediction phase covers the subsequent $(N/2)$ steps, where the velocity of the predictable actions is linearly extrapolated using their earlier velocity and regressed acceleration. Conversely, other unpredictable actions undergo full diffusion inference to maintain accuracy. Due to velocity linearity, action field and KV cache of the predictable actions are extrapolated following a quadratic pattern. (3) A correction step is executed at the final iteration, where a full action expert inference is applied to all actions. The correction step mitigates accumulated extrapolation errors and ensures trajectory convergence.

B. Flow Predictor Architecture

As depicted in Fig. 7, the flow predictor circuit comprises three primary functional modules: (1) The fitting module performs element-wise accumulation of the statistics of the action, velocity and KV tokens throughout the fitting phase. The required statistics are represented as unweighted, t -weighted, and t^2 -weighted sums, thereby eliminating the storage of the original velocity and action sequences. Based on these accumulated statistics, the R^2 value and the quadratic regression coefficients are computed for each element. (2) The classification module identifies predictable action tokens through element-wise comparison of the velocity R^2 values. Specifically, an action token is classified as predictable when more than half of its velocity dimensions exhibit R^2 values exceeding the predefined threshold of 0.5. Once an action token is identified as predictable, the corresponding velocity token and KV token are jointly assigned to the

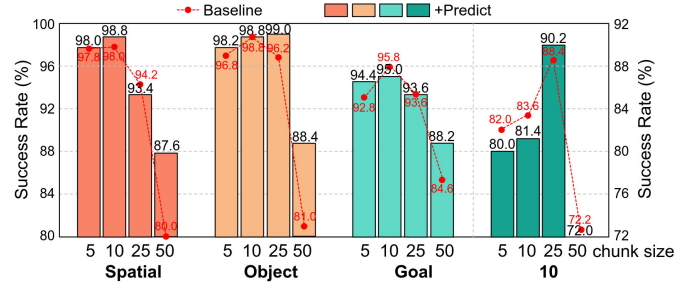


Fig. 8. Comparison of accuracy with the flow predictor on the LIBERO benchmark.

predictable set for subsequent extrapolation. (3) The prediction module extrapolates the predictable tokens using the fitted coefficients. Specifically, the velocity elements are linearly extrapolated, while the corresponding action field and KV cache elements are predicted following a quadratic trajectory.

Fig. 8 presents the success rate evaluation of the proposed flow predictor on the LIBERO benchmark using an INT8-quantized version of π_0 under synchronous planning. The policy generates a sequence of 50 candidate actions, with chunk sizes ranging from 5 to 50 for robotic execution. The baseline corresponds to the original diffusion process without prediction using 10 diffusion steps, whereas the proposed predictor reduces the number of full diffusion steps to 5. In the relatively simple task suites, including LIBERO-Spatial, LIBERO-Object, and LIBERO-Goal, the proposed predictor introduces only a success-rate degradation of 0.8% in the worst case. For the more challenging suite LIBERO-10, the maximum success-rate degradation remains within 2.2%. Moreover, the predictor achieves higher success rates than the baseline under several configurations, particularly at chunk sizes of 25 and 50. This improvement may arise because velocity extrapolation smooths the action fluctuations in the iterative updates. During the later diffusion steps, such fluctuations accumulate and propagate along the action sequence, leading to increasingly pronounced action errors toward the end of chunks. By suppressing these fluctuations, the predictor improves the accuracy of later actions in the chunk relative to the baseline. We further observe that unpredictable actions are primarily concentrated near the beginning of each chunk, with few occurring at later positions, which further supports the interpretation. Overall, these results demonstrate that the proposed predictor effectively reduces redundant diffusion iterations while maintaining comparable action-generation accuracy, making it suitable for real-time robotic deployment. The limited degradation across the four LIBERO suites further indicates that the flow-velocity regularity exploited by the predictor remains effective across tasks of varying complexity. Since this regularity originates from the rectified-flow formulation (Eq. 2, Eq. 3) rather than π_0 -specific operations, the proposed method also has the potential to extend to other rectified-flow-based policies.

Fig. 9 presents the ablation results for the flow predictor. Fig. 9(a) evaluates the effects of runtime precision, the number of predicted diffusion steps, and the final correction step. The results show that the policy maintains a high success rate when

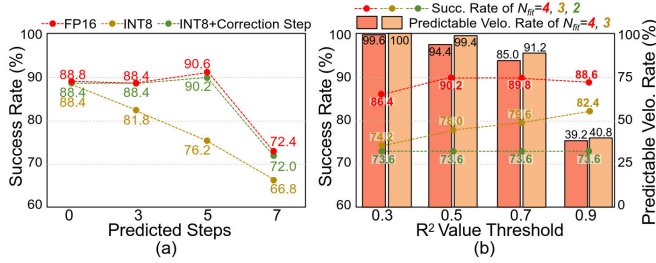


Fig. 9. Ablation study of the flow predictor: (a) effects of runtime precision, the number of predicted diffusion steps, and the final correction step; (b) effect of the R^2 threshold.

no more than five diffusion steps are predicted. INT8 quantization introduces negligible accuracy degradation, whereas the final correction step effectively mitigates the accumulated extrapolation errors. Accordingly, the number of predicted diffusion steps is set to five, followed by a final correction step, thereby maximizing the number of skipped diffusion iterations while preserving policy accuracy. Fig. 9(b) evaluates the effects of the fitting-step counts and the R^2 threshold, with the number of predicted steps fixed at 5. The evaluated fitting-step counts are 4, 3, and 2, corresponding to diffusion steps {0, 1, 2, 3}, {1, 2, 3}, and {2, 3}, respectively. For 3 and 4 fitting steps, increasing the R^2 threshold improves the success rate but substantially reduces the proportion of velocities classified as predictable. With only 2 fitting steps, the two velocity samples directly determine the linear extrapolation, resulting in a predictable-action rate of 100%. To balance prediction coverage and policy accuracy, the R^2 threshold is set to 0.5. Moreover, reducing the fitting-step counts increases the risk of overfitting to the limited velocity samples, leading to degraded policy accuracy. Therefore, the number of fitting steps is set to 4.

Fig. 10 compares the average inference latency and energy consumption with and without the flow predictor. By bypassing redundant diffusion steps, the flow predictor reduces the latency and energy consumption by factors of $1.79\times$ and $1.77\times$, respectively. In the observed best case, all 50 actions are classified as predictable, yielding a $1.98\times$ inference speedup and a $1.98\times$ energy-efficiency improvement. In the observed worst case, 8 of the 50 actions are classified as unpredictable and retain full action-expert computation; nevertheless, the accelerator still achieves a $1.66\times$ inference speedup and a $1.67\times$ energy-efficiency improvement. Consequently, the predictor enables real-time and energy-efficient edge-side action planning.

V. ADAPTIVE REAL-TIME CHUNKING SCHEDULER

A. Dual-Mode RTC Strategy

RTC enables asynchronous action planning by overlapping action expert inference with the execution of the preceding action chunk. The RTC strategy [11] is adopted to facilitate asynchronous action planning by decoupling the action expert inference from robotic action execution completion. However, the original RTC formulation (Eq. 4) relies on gradient-based flow guidance and requires backward propagation at

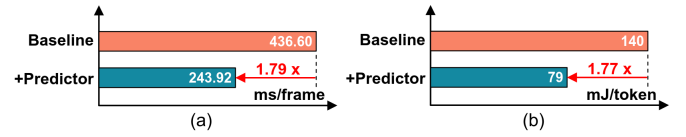


Fig. 10. Comparison of the average inference latency and energy consumption with the flow predictor.

each diffusion step, incurring substantial computational and memory-access overhead.

To derive a lightweight flow-guidance formulation, we leverage the smooth linear trajectory induced by rectified flow. As shown in Eq. 4, the estimated final action field satisfies $\hat{\mathbf{a}}_t \approx \mathbf{x}_t + (1-t)\mathbf{v}_t$ in the original RTC algorithm, while the target field can be further expressed as $\mathbf{a}^* \approx \mathbf{x}_t + (1-t)\mathbf{v}_t^*$ in the same form. Subtracting the two relations directly yields a new feasible velocity correction form (Eq. 5):

$$\Delta \mathbf{v}_t = \mathbf{v}_t^* - \mathbf{v}_t \approx \frac{\mathbf{a}^* - \hat{\mathbf{a}}_t}{1-t} \quad (5)$$

The resulting gradient and rectified guidance modes are formulated in Eq. 6. This target-directed correction replaces gradient computation with lightweight element-wise vector operations in suitable execution phases, thereby substantially reducing the computational and memory-access overhead incurred by RTC guidance. Importantly, according to the above derivation, the velocity correction in rectified guidance constitutes an feasible alternative solution derived from the rectified-flow trajectory, rather than an approximation of the gradient-guidance correction. Moreover, although the factor $\frac{1}{1-t}$ may amplify the velocity correction as t approaches 1, its contribution to the flow-field update appears as the product $\Delta \mathbf{v}_t \Delta t$ in $\mathbf{x}_{t+\Delta t}$ (Eq. 2). In the implementation, the combined factor $\frac{\Delta t}{1-t}$ bounded above by 1.0 is evaluated before applying the correction, avoiding an excessively large intermediate value and improving numerical stability.

$$\Delta \mathbf{v}_t^{(i)} = \begin{cases} r_t w_i (\mathbf{a}^{*(i)} - \hat{\mathbf{a}}_t^{(i)})^T \frac{\partial \hat{\mathbf{a}}_t^{(i)}}{\partial \mathbf{x}_t^{(i)}}, & \text{(gradient guid.)} \\ w_i \frac{\mathbf{a}^{*(i)} - \hat{\mathbf{a}}_t^{(i)}}{1-t}, & \text{(rectified guid.)} \end{cases} \quad (6)$$

The adaptive RTC strategy selects between the two modes according to the robotic motion state: (1) Gradient guidance retains the original RTC formulation and performs complete backward propagation at each diffusion step. This mode provides higher action accuracy and is therefore applied during precision-sensitive phases, such as object reaching, alignment, and grasping. (2) Rectified guidance applies the proposed target-directed velocity correction through lightweight vector operations. This mode is suitable for motion phases with relaxed accuracy requirements, such as free-space end-effector motion, where a broad set of feasible action trajectory solutions is available. By dynamically switching between the two modes, the adaptive RTC strategy reduces gradient computation with limited action-quality degradation while improving overall energy efficiency.

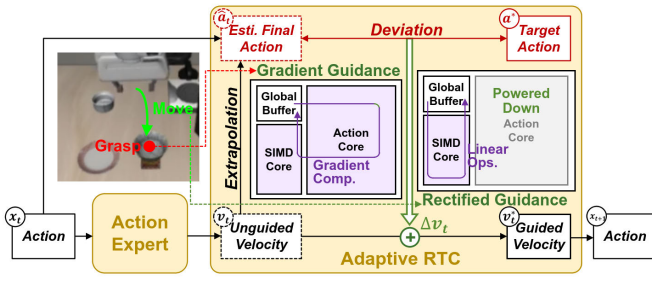


Fig. 11. The adaptive dataflow of RTC based on gradient guidance and rectified guidance.

B. Dataflow for Gradient and Rectified Guidance

The dataflows for the two RTC modes are illustrated in Fig. 11. The CPU acquires the robotic motion state and provides it to the adaptive RTC scheduler, which selects the guidance mode accordingly. Specifically, the scheduler employs an aggressive selection scheme using the gripper state reported by the CPU. When the relative gripper displacement exceeds 1% of its full motion range, the scheduler classifies the current operation as a precision-sensitive manipulation phase, such as grasp initiation or contact-angle adjustment, and activates gradient guidance. Otherwise, rectified guidance is applied to motion phases with relaxed accuracy requirements. Guidance-mode selection is performed independently for each action chunk. At the beginning of each chunk, the scheduler evaluates the latest robotic state and selects the corresponding RTC mode. The selected mode remains fixed throughout all diffusion steps associated with that chunk.

Prior to the flow guidance process, the action fields, velocity fields, and auxiliary parameters are stored in the global buffer. The gradient guidance is activated during complex or delicate operations. In this mode, the action core serves as the primary computational engine for gradient tensor multiplications during backward propagation. Gradient operand tensors are prefetched from DDR into the global buffer, while transposed weight or activation operands are loaded into the weight memory near the engine array. The operands entering the action core undergo online quantization, matrix multiplication with weights or activations, and dequantization to produce gradient results for subsequent propagation. The rectified guidance is enabled when the precision requirement is relaxed. In this mode, the scheduler activates only the SIMD core, while the action core enters a sleeping state with its processing arrays and associated memories powered down. This mode yields substantial energy savings without compromising overall task success.

Fig. 12 compares the task success rates achieved by different RTC strategies. In the asynchronous action planning, the absence of RTC causes a substantial success rate degradation, indicating that RTC is essential for mitigating the accuracy loss caused by the observation gap. Moreover, the proposed adaptive RTC strategy incurs only a 1.0% success rate degradation relative to the original RTC algorithm, demonstrating its ability to preserve asynchronous planning accuracy to a comparable level. LIBERO-10 exhibits the largest success rate degradation among the evaluated suites, consistent with its greater task

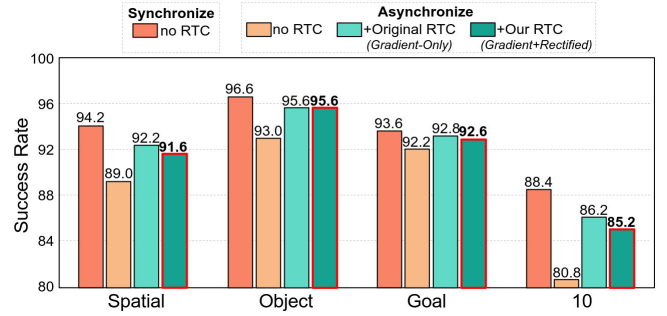


Fig. 12. Comparison of accuracy with the RTC strategies on the LIBERO benchmark.

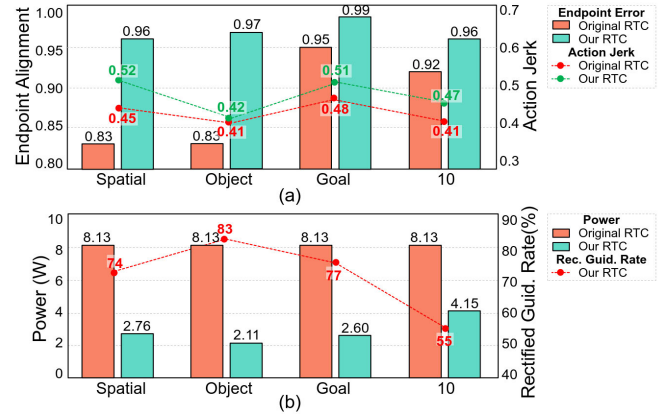


Fig. 13. Comparison of (a) endpoint alignment and action jerk across diverse tasks; and (b) average power and rectified-guidance utilization rate under different RTC strategies.

complexity and multi-stage sub-behavior sequences. Failure-case analysis indicates that errors frequently occur during transitions between consecutive sub-behaviors, such as switching from “turn on the stove” to “put the moka pot on it.” During these transitions, the adaptive RTC strategy may induce hesitation and repeated corrective actions, eventually preventing successful task completion. These results demonstrate that dynamically selecting between gradient guidance and rectified guidance according to the robotic state incurs negligible accuracy degradation while substantially reducing computational overhead. Additional studies on endpoint alignment and action jerk are presented in Fig. 13(a). Endpoint alignment quantifies the consistency between the overlapped actions of consecutive chunks and is measured by the average cosine similarity across the evaluated tasks. The adaptive RTC mechanism improves endpoint alignment because rectified guidance directly steers the flow field toward the target field at the final diffusion step, whereas gradient guidance applies an iterative gradient-based heuristic correction without a mandatory endpoint-alignment constraint. Action jerk quantifies the continuity and smoothness of the generated action sequence and is measured by the average magnitude of the second-order difference. The adaptive RTC mechanism introduces a slight increase in action jerk because rectified guidance constructs the flow-field trajectory primarily based on the rectified-flow prior and the target field, with less emphasis on action smoothness than the original

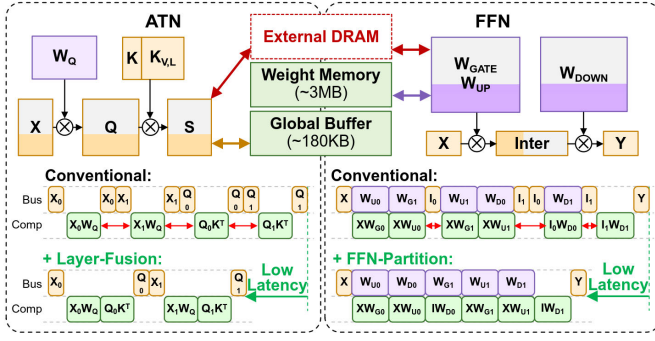


Fig. 14. The excessive weights and intermediate activations in Transformer blocks.

gradient guidance. Nevertheless, the resulting action jerk of adaptive RTC remains close to that of the original RTC while yielding lower endpoint error.

As illustrated in Fig. 13(b), the proposed adaptive RTC scheduler assigns rectified guidance to 55%–83% of the RTC operations across the LIBERO suites. The simpler task suites exhibit higher rectified-guidance utilization than the more complex LIBERO-10 suite because a larger proportion of their motion phases are amenable to lightweight guidance. Consequently, the adaptive RTC scheduler achieves a $1.96 \times -3.85 \times$ reduction in average RTC-phase power across the evaluated suites. These results demonstrate that the adaptive RTC scheduler improves energy efficiency while maintaining task accuracy.

VI. MEMORY-EFFICIENT DATAFLOW OPTIMIZATION

In conventional DiT-based action expert inference, layers are executed sequentially, requiring full weight matrices and token sequences to be resident on-chip for matrix multiplication. This design introduces substantial storage overhead for both model weights and intermediate activations, as shown in Fig. 14. Most weights and activations must be stored in off-chip DDR memory during inference. This memory-hierarchy constraint prohibits full on-chip data residency and incurs frequent off-chip data transfers, resulting in substantial processing stalls and performance degradation.

A. Tile-Wise Layer-Fusion Dataflow

To alleviate the aforementioned memory-access bottlenecks, a hierarchical layer-fusion scheme is proposed that partitions the token sequence into tiles and enables tile-wise pipelined processing across fused layers. The activation tiles are carefully sized to fit within the limited on-chip memory capacity, with each tile processed independently. To enable independent tile-level processing, consecutive layers are fused into a unified pipeline that operates in a tile-by-tile manner, rather than processing the entire token sequence before proceeding to subsequent layers. During tile processing, intermediate activations are consumed immediately upon generation, thereby minimizing stall-induced data movement.

During forward propagation, cross-attention requires global access to tokens, necessitating aggregation of KV across all

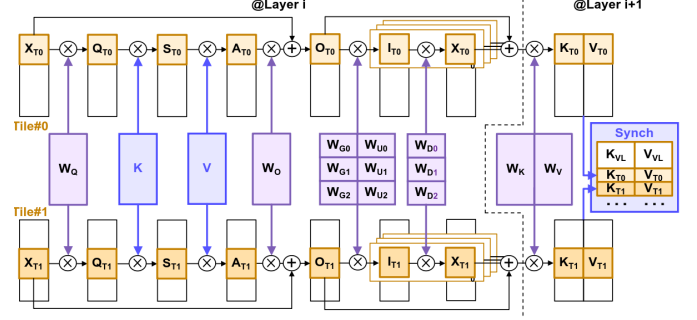


Fig. 15. Scheduled dataflow of forward propagation.

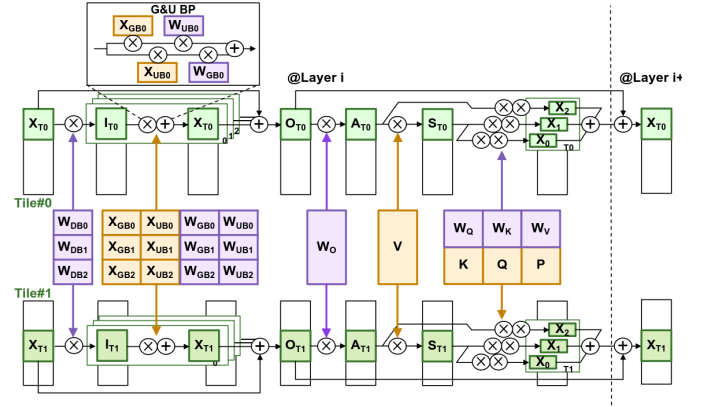


Fig. 16. Scheduled dataflow of backward propagation.

tiles before computation. To accommodate this dependency, layers within a Transformer block are fused into a unified pipeline, with fusion boundaries determined by KV synchronization points. As illustrated in Fig. 15, the proposed scheme jointly fuses the non-KV-projection layers of block i and the KV-projection layers of block $(i + 1)$, thereby forming a continuous inter-block pipeline between two consecutive cross-attention layers. Token tiles are streamed from DDR at the pipeline input, processed sequentially, and written back after completing the fused stages. Meanwhile, the KV cache of block $(i + 1)$ is retained on-chip throughout the processing of all tiles, enabling the global KV cache to be fully synchronized on-chip. Throughout this process, intermediate activations and residual tokens remain on-chip, significantly reducing memory traffic and pipeline stalls. When the gradient guidance is enabled, intermediate activations required for subsequent backward propagation are offloaded to DDR, which is entirely overlapped with the forward computation and causes no additional latency overhead.

During backward propagation under RTC-enabled inference, the tiling and layer-fusion scheme follows a similar structure. As shown in Fig. 16, token gradients are partitioned into tiles along the sequence dimension. Unlike forward propagation, gradient computation does not involve cross-token dependencies, eliminating the need for global synchronization. For each tile, all layers within a Transformer block are fused into a unified backward pipeline. The gradient tile first traverses the FFN stage and then proceeds to the attention stage, involving operations with transposed weights and stored

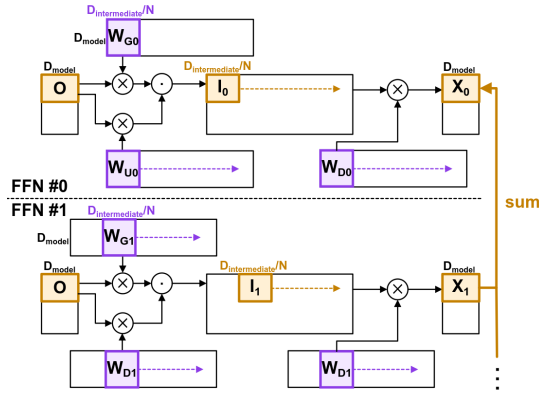


Fig. 17. Scheduled dataflow of FFN-partition in forward propagation.

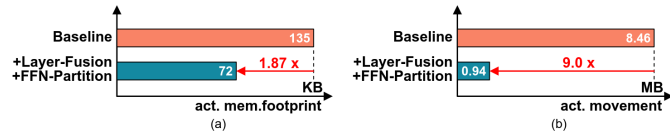


Fig. 18. Comparison of (a) activation memory footprint. (b) activation movement with layer-fusion and FFN-partition dataflow.

intermediate activations. Throughout the process, the entire gradient tile is retained on-chip, thereby eliminating pipeline stalls caused by external memory accesses.

B. FFN-Partition Dataflow

In FFN layers, the gate-, up-, and down-projection weight matrices often exceed the available on-chip memory capacity, requiring frequent off-chip accesses and introducing pipeline stalls. To address this limitation, each FFN layer is partitioned into a sequence of smaller sublayers, decomposing both weight matrices and intermediate activations into compact submatrices that fit entirely within on-chip memory. This partitioning scheme operates in conjunction with the global layer-fusion mechanism, as illustrated in Fig. 15 and Fig. 16.

As depicted in Fig. 17, during forward propagation, FFN projections are partitioned along the intermediate dimension. In each sublayer, input tokens are processed by the partitioned gate- and up-projection, producing intermediate features of reduced dimension. These features are then consumed by the corresponding portion of down-projection to generate partial results. Subsequent sublayers operate on the same input tokens, incrementally accumulating partial results until the final FFN output is produced. During backward propagation, the same partitioning scheme is applied to gradient computation. Gradients are processed sequentially across sublayers and accumulated to preserve computational correctness. This scheme eliminates the demand for storing oversized data on-chip and significantly mitigates stalls for external memory accesses.

In the implementation, the token sequence is partitioned into two tiles, while each FFN layer is decomposed into four sublayers. Under this configuration, the proposed hierarchical layer-fusion and FFN-partition scheme reduces the peak memory footprint by 1.87 $\times$ and decreases stall-induced

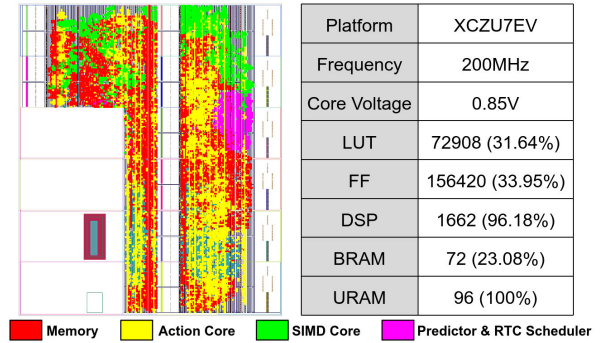


Fig. 19. FPGA implementation layout and resource utilization of the proposed accelerator.

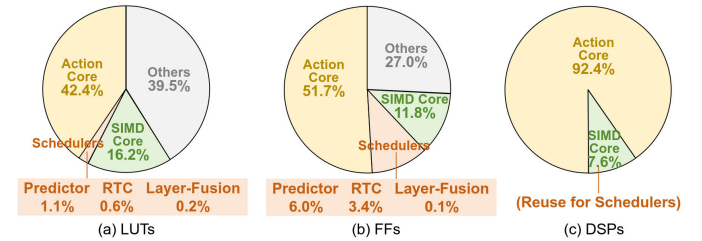


Fig. 20. Breakdown of accelerator resource utilization: (a) LUTs, (b) FFs, and (c) DSPs.

intermediate activation movement by 9.0 $\times$. These improvements enable efficient action planning under strict on-chip resource constraints, as illustrated in Fig. 18. Notably, the layer-fusion and FFN-partition dataflow applies computation-preserving partitioning and scheduling, thereby retaining the original policy inference accuracy.

VII. IMPLEMENTATION AND EVALUATION

A. Implementation Setup

1) *FPGA Prototype*: The proposed FPGA-based robotic action planning accelerator is implemented on an AMD Xilinx XCZU7EV FPGA platform. Fig. 19 presents the layout and on-chip resource utilization details. The design operates at 200 MHz under a voltage supply of 0.85 V. The action core and SIMD core are mapped mainly to DSP slices for INT8 matrix operations and FP16 vector operations, respectively. The weight memory is implemented using UltraRAMs, while the global buffer and input/output activation buffers are implemented using Block RAMs. The implementation utilizes 72.9k LUTs, 156.4k FFs, 1662 DSPs, 72 BRAMs, and 96 UltraRAMs. Fig. 20 presents the breakdown of accelerator resource utilization. The flow predictor, RTC scheduler, and layer-fusion scheduler collectively account for only 1.9% of the LUT utilization and 0.5% of the FF utilization. Their compute-intensive operations are mapped onto the SIMD core, thereby enabling efficient reuse of existing DSP resources. Moreover, all URAM resources are allocated to the weight memory of the action core, whereas the BRAM resources are allocated to the global buffer and the input/output buffers of the action core, as illustrated in Fig. 4. These results demonstrate

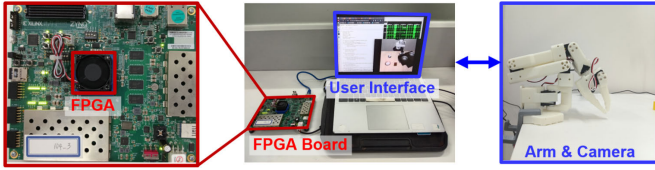


Fig. 21. Simulation and SO-101 real-robot demonstration setup.

that the proposed optimization modules provide the reported performance improvements with minimal resource overhead.

2) *Model and Policy Configuration*: We evaluate the accelerator using the action expert of the π_0 VLA policy [2]. The action expert is a flow-matching-based DiT with approximately 300 million parameters and employs a 10-step diffusion process. Each inference generates a sequence of 50 actions, of which the first 25 are executed and the remainder are truncated. Linear operations employ INT8 nearest-value quantization, whereas the remaining operations retain FP16 precision. The flow predictor uses the first four diffusion steps for velocity fitting, predicts the subsequent five steps, and retains the final step for correction. In the synchronous mode, inference begins only after the preceding action chunk has been fully executed, with RTC disabled. In the asynchronous mode, inference begins while the final five actions of the preceding chunk are being executed, with RTC applied to these five overlapping actions.

3) *Simulation on LIBERO Benchmark*: We evaluate the proposed system on four LIBERO benchmark suites: LIBERO-Spatial, LIBERO-Object, LIBERO-Goal, and LIBERO-10. We select 10 tasks from each suite and conduct 50 single-trial evaluation episodes with independently randomized environment initializations for each task. LIBERO-Spatial, LIBERO-Object, and LIBERO-Goal mainly comprise relatively simple, short-horizon manipulation tasks, such as tabletop pick-and-place and object rearrangement, which typically involve a single-stage operation. In contrast, LIBERO-10 contains more complex tasks, including cookware manipulation in kitchen scenes and object sorting and relocation in living-room scenes, thereby requiring longer-horizon action planning. The balanced selection of tasks across the four suites covers diverse embodied-AI application scenarios, manipulation complexities, and action-sequence lengths. Moreover, the randomized evaluation episodes improve the representativeness of the benchmark for assessing both action fidelity and execution efficiency. The LIBERO environments are implemented in the MuJoCo physics simulator [25], which supports the randomly initialized placement of digital object assets and the physically realistic simulation of robot-object interactions. Owing to randomness in environment initialization and physics-based simulation of visual observation and action execution, together with finite-precision quantization and truncation errors, the measured success rate exhibits an uncertainty of approximately 1%. All system-level evaluations and ablation studies adopt this benchmark configuration.

4) *Real-Robot Demonstration*: In addition to simulation-based evaluation, we validate the proposed acceleration system on a real-robot using the experimental setup shown in

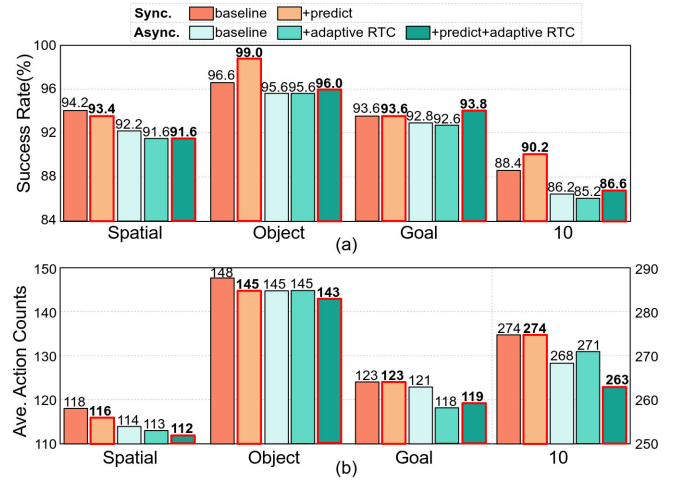


Fig. 22. Comparison of π_0 action expert inference on the accelerator in (a) task success. (b) average actions.

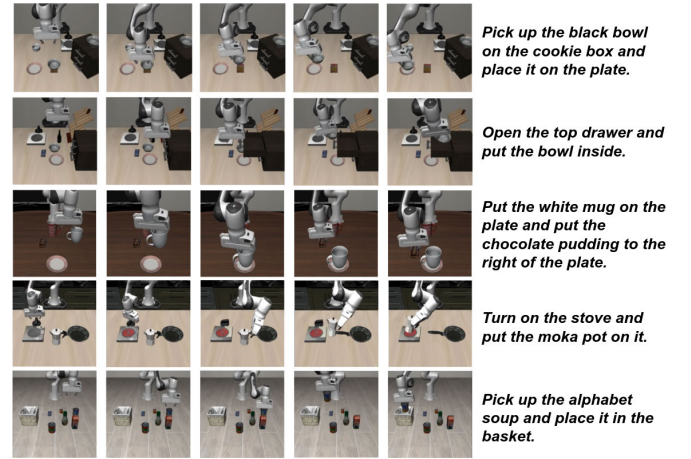


Fig. 23. Simulation results across various instructions on diverse scenes.

Fig. 21. The real-robot platform employs an SO-101 6-joint robotic arm with a gripper [26], controlled through the open-source LeRobot library [29]. Visual observations and language instructions are processed by the server-side VLM, while the FPGA accelerator performs edge-side action expert inference to generate action chunks for execution by the robot arm.

B. Accuracy, Latency, and Energy Efficiency Evaluation

1) *Task Success Rate*: Fig. 22(a) presents the task success rates for the π_0 action expert inference under different optimization settings. In the synchronous action planning, the baseline employs the action expert without flow prediction. Enabling the flow predictor limits the maximum success-rate loss to 0.8% across the evaluated benchmarks, indicating that the predictor largely preserves task accuracy while improving inference throughput. In the asynchronous action planning, the baseline employs conventional RTC without flow prediction. With both the flow predictor and adaptive RTC scheduler enabled, the proposed accelerator incurs only a 0.6% success-rate degradation relative to this baseline in the worst case, demonstrating that the proposed optimizations maintain action fidelity while supporting asynchronous execution.

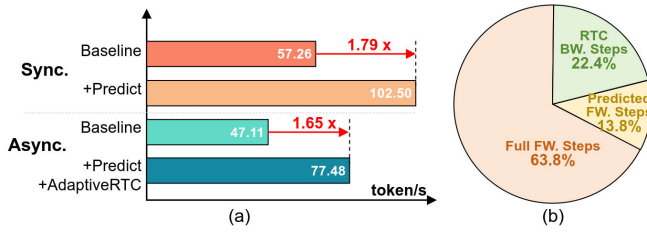


Fig. 24. (a) Comparison of π_0 action expert inference latency on the accelerator. (b) Inference latency breakdown by processing phase.

TABLE I
COMPARISON OF DIFFERENT COMPUTING DEVICES

Device	Tech	Number of Cores	SRAM (MB)	Theoretical Peak Perf.	Peak Power (W)
RTX 3090 [30]	SamSung 8 nm	328 Tensor Cores	10.25	35.6 TFLOPS	350
AGX Orin 64GB [31]	SamSung 8 nm	64 Tensor Cores	4.10	5.3 TFLOPS	60
This Work	TSMC 16 nm (FPGA)	16 PE Array	3.44	921.6 GOPS	10

Fig. 22(b) further reports the average number of executed actions per task. The proposed flow predictor and RTC scheduler do not increase the required action count, suggesting that the optimized inference does not introduce additional motion inefficiency. Fig. 23 provides representative simulation trajectories across different language instructions and manipulation scenes, further confirming that the system maintains high-level planning capability and primitive action accuracy.

2) *Control Rate and Throughput*: The proposed accelerator achieves a peak performance of 903.2 GOPS operating at 200 MHz and 0.85 V. Fig. 24(a) compares the effective control rates under different execution modes. In the synchronous action planning, enabling the flow predictor increases the control rate from 57.26 Hz to 102.50 Hz, corresponding to a $1.79\times$ improvement. In the asynchronous action planning, the RTC mechanism introduces additional backward-propagation overhead for flow guidance; nevertheless, with both the flow predictor and adaptive RTC scheduler enabled, the system sustains a 77.48-Hz control rate. This rate exceeds the 50-Hz real-time control requirement while enabling overlap between action generation and robot execution. Fig. 24(b) presents the inference latency breakdown by processing phase. Full action expert forward steps dominate the inference latency, accounting for 63.8% of the total. Predicted forward steps and RTC backward steps account for 13.8% and 22.4%, respectively. Their smaller proportions arise because complete forward computation in the predicted steps is restricted to unpredictable action tokens, whereas RTC backward computation is applied only to overlapped action tokens.

3) *Energy Efficiency*: Fig. 25(a) evaluates the energy efficiency of π_0 action expert inference, with the results obtained from post-implementation power estimation using switching activity from gate-level simulation and AMD Xilinx Power Estimator (XPE). The FPGA prototype consumes 8.13 W and achieves an energy efficiency of 111.09 GOPS/W. In the synchronous mode, the flow predictor reduces the energy efficiency from 140 mJ/token to 79 mJ/token, corresponding to a $1.77\times$ improvement. In the asynchronous mode, the combined flow predictor and adaptive RTC scheduler reduce

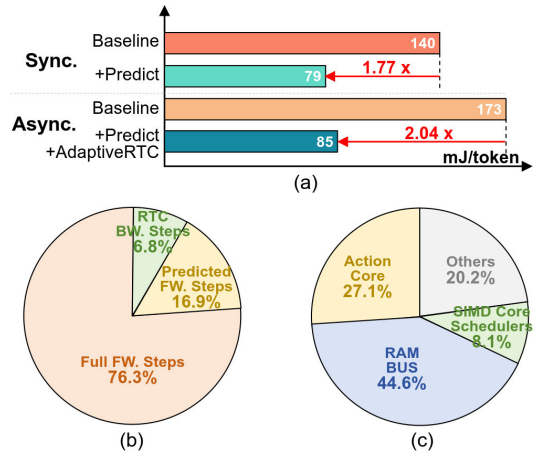


Fig. 25. (a) Comparison of π_0 action expert energy efficiency on the accelerator. (b) Energy consumption breakdown by processing phase. (c) Energy consumption breakdown by hardware component.

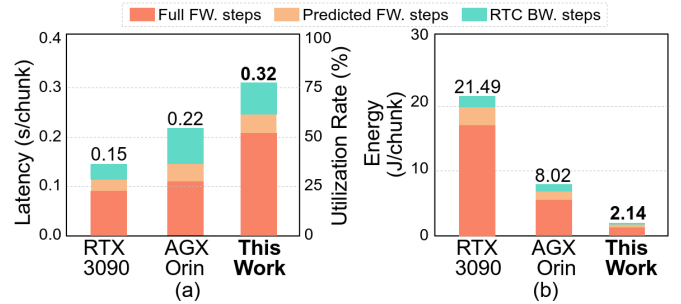


Fig. 26. Comparison and breakdown of average latency and energy consumption per action chunk for π_0 action expert inference on the proposed accelerator and GPU platforms.

the energy efficiency from 173 mJ/token to 85 mJ/token, achieving a $2.04\times$ improvement. These results show that the proposed predictor and RTC scheduler reduce both redundant computation and synchronization overhead, enabling energy-efficient edge-side planning. Fig. 25(b) presents the energy consumption breakdown by processing phase. Full action expert forward steps account for the largest fraction, at 76.3%, whereas predicted forward steps account for 16.9%. Adaptive RTC limits the contribution of RTC guidance to 6.8% by selecting lightweight rectified guidance during eligible robotic motion phases. Fig. 25(c) presents the energy consumption breakdown by hardware modules. RAM accesses and bus transactions form the largest component, accounting for 44.6% of the total energy consumption. The action core contributes 27.1%, representing the largest module among the computational circuits, while the SIMD core along with the predictor and schedulers contributes 8.1%. The remaining 20.2% arises from the processing system (PS), clock and reset networks, and peripheral control logic. This breakdown indicates that data movement and the action core dominate the overall energy consumption.

C. Comparison With GPUs and Prior Works

1) *Comparison With GPUs*: Fig. 26 compares and breaks down the average latency and energy consumption of π_0

TABLE II
COMPARISON WITH PRIOR WORKS

	TCASI'23 [14]	FPL'24 [15]	ASP-DAC'25 [16]	TCASI'25 [17]	This Work
FPGA Platform	XCZU9EG	XCZU9EG	XCU200	XCU50	XCZU7EV
Application	Vision Understanding	Vision Generation	Vision Generation	Vision Understanding	Robotic Action Planning
Model	ViT-S (22M) ViT-B (86M)	SD1.5 UNet (860M)	SD1.5 UNet (860M) SDXL UNet (2.6B)	Swin-T (28M) Swin-S (50M)	π_0 action expert (300M)
Model architecture	Transformer Encoder	Diffusion Transformer+CNN	Diffusion Transformer+CNN	Transformer Encoder	Diffusion Transformer
Precision	INT8	INT4/INT8	INT8	INT8	INT8
Frequency (MHz)	300	250	-	170	200
kLUTs/DSPs	144/1268	221/806	783/5056	472/2164	73/1662
Power (W)	29.6	10.0**	18.2**	18.4	8.13
Throughput (GOPS)	762.7	332.5	1395*	830.26	903.2
Resource Efficiency (GOPS/kDSP)	573.1	412.5	275.9*	383.7	543.4
Energy Efficiency (GOPS/W)	25.77	33.25	76.65*	45.12	111.09

* The reported value is unavailable in the original work and is therefore estimated using the SD1.5 UNet workload.

** The power value is obtained from direct hardware measurement as reported in the original work.

action expert inference on the proposed accelerator and two GPU platforms. The corresponding platform configurations are summarized in Table I. The comparison includes the NVIDIA RTX 3090 [30], representing a high-performance datacenter-oriented GPU platform, and the NVIDIA Jetson AGX Orin 64 GB [31], representing an embedded edge-side GPU platform. For consistency, flow prediction, adaptive RTC, and the corresponding layer-fusion optimization are enabled on all three platforms. In terms of inference latency, the proposed accelerator generates each action chunk in 0.32 s, whereas the RTX 3090 and Jetson AGX Orin achieve shorter processing latencies. In terms of energy consumption, the proposed accelerator requires 2.14 J per action chunk at an average power of 8.13 W. By comparison, the RTX 3090 and Jetson AGX Orin consume $10.04\times$ and $3.75\times$ the energy per action chunk, respectively, with average powers of 176.6 W and 53.0 W. Thus, although the GPU platforms achieve lower absolute latency, the proposed accelerator delivers competitive throughput with substantially lower energy consumption, demonstrating its advantage for edge-side action expert deployment.

2) *Comparison With Prior Works:* Table II compares the proposed design with representative FPGA-based Transformer accelerators [14], [15], [16], [17]. Given the limited availability of directly comparable FPGA accelerators for robotic action planning, the comparison includes designs targeting vision understanding and vision generation. Although these accelerators address different applications, their ViT, Swin Transformer, and Stable Diffusion workloads contain computation-intensive Transformer or diffusion kernels and span model scales relevant to the 300M-parameter π_0 action expert. For a fair comparison, the reported metrics are evaluated at the granularity of a single ViT inference or a single diffusion step, depending on the workload. Power values obtained from direct hardware measurements are identified in the table. Implemented on the XCZU7EV FPGA, the proposed design achieves 903.2 GOPS at 8.13 W, corresponding to an energy efficiency of 111.09 GOPS/W and a DSP-normalized resource efficiency of 543.4 GOPS/kDSP. Among the designs listed in Table II, the proposed accelerator achieves

the highest energy efficiency, representing improvements of $1.45\times$ – $4.31\times$ over the prior works, together with a relatively high throughput and DSP-normalized resource efficiency. Although ASP-DAC'25 [16] reports a higher throughput of 1395 GOPS using DSP58 slices, the proposed design is implemented with DSP48 slices and achieves $1.98\times$ higher resource efficiency and $1.45\times$ higher energy efficiency. These results demonstrate the benefits of co-designing the accelerator around predictable flow-matching inference, adaptive RTC execution, and a memory-efficient dataflow.

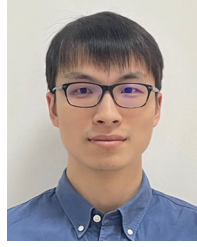
VIII. CONCLUSION

This work presents an FPGA accelerator for diffusion-based VLA action planning in embodied AI systems. The proposed design integrates a flow predictor, an adaptive RTC scheduler, and a memory-efficient dataflow to address the latency, synchronization overhead, and memory bottlenecks of edge-side action experts. Implemented for the π_0 model, the accelerator achieves 903.2 GOPS and 111.09 GOPS/W, sustaining a 77.48-Hz control rate at 85 mJ/token under asynchronous execution. Experimental results on the LIBERO benchmark and real-robot deployment on an SO-101 platform demonstrate that the proposed design enables real-time and energy-efficient robotic action planning on resource-constrained edge devices.

REFERENCES

- [1] Y. Liu et al., "Aligning cyber space with physical world: A comprehensive survey on embodied AI," *IEEE/ASME Trans. Mechatronics*, vol. 30, no. 6, pp. 7253–7274, Dec. 2025.
- [2] K. Black et al., " π_0 : A vision-language-action flow model for general robot control," 2024, *arXiv:2410.24164*.
- [3] Q. Li et al., "CogACT: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation," 2024, *arXiv:2411.19650*.
- [4] G. Lu et al., "VLA-RL: Towards masterful and general robotic manipulation with scalable reinforcement learning," 2025, *arXiv:2505.18719*.
- [5] K. Black et al., " π_0 .5: A vision-language-action model with open-world generalization," in *Proc. 9th Conf. Robot Learning (CoRL)*, vol. 305, Sep. 2015, pp. 17–40.
- [6] NVIDIA et al., "GR00T N1: An open foundation model for generalist humanoid robots," 2025, *arXiv:2503.14734*.
- [7] B. Zitkovich et al., "RT-2: Vision-language-action models transfer web knowledge to robotic control," in *Proc. 7th Conf. Robot Learn.*, vol. 229, Nov. 2023, pp. 2165–2183.

- [8] W. Peebles and S. Xie, "Scalable diffusion models with transformers," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 4172–4182.
- [9] Y. Huang et al., "Dadu-corki: Algorithm-architecture co-design for embodied AI-powered robotic manipulation," in *Proc. 52nd Annu. Int. Symp. Comput. Archit.*, Jun. 2025, pp. 327–343.
- [10] Y. Lipman et al., "Flow matching for generative modeling," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023, pp. 1–28.
- [11] K. Black, M. Y. Galliker, and S. Levine, "Real-time execution of action chunking flow policies," 2025, *arXiv:2506.07339*.
- [12] S. Zhang, X. Chen, C. Peng, H. Yang, Y. Liu, and H. Jia, "A 94Hz inference and 7.4mJ/Epoch fine-tune edge SoC for diffusion-based robot manipulation with speculation and disturbance enhancement," in *Proc. Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, Jun. 2025, pp. 1–3.
- [13] Y. Dai et al., "ActionFlow: A pipelined action acceleration for vision language models on edge," 2025, *arXiv:2512.20276*.
- [14] M. Huang, J. Luo, C. Ding, Z. Wei, S. Huang, and H. Yu, "An integer-only and group-vector systolic accelerator for efficiently mapping vision transformer on edge," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 12, pp. 5289–5301, Dec. 2023.
- [15] G. Yang et al., "SDA: Low-bit stable diffusion acceleration on edge FPGAs," in *Proc. 34th Int. Conf. Field-Programmable Log. Appl. (FPL)*, Sep. 2024, pp. 264–273.
- [16] H. Zhou et al., "Deploying diffusion models with scheduling space search and memory overflow prevention based on graph optimization," in *Proc. 30th Asia South Pacific Design Autom. Conf.*, Jan. 2025, pp. 170–176.
- [17] Q. Dong, S. Zhang, and Z. Wang, "An efficient window-based vision transformer accelerator via mixed-granularity sparsity," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 72, no. 9, pp. 4751–4764, Sep. 2025.
- [18] J. Liu et al., "FlightVGM: Efficient video generation model inference with online sparsification and hybrid precision on FPGAs," in *Proc. ACM/SIGDA Int. Symp. Field Program. Gate Arrays*, Feb. 2025, pp. 2–13.
- [19] M. Huang et al., "EdgeLLM: A highly efficient CPU-FPGA heterogeneous edge accelerator for large language models," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 72, no. 7, pp. 3352–3365, Jul. 2025.
- [20] P. Dong et al., "HeatViT: Hardware-efficient adaptive token pruning for Vision Transformers," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, Feb. 2023, pp. 442–455.
- [21] S. Tang et al., "Diff-acc: An efficient FPGA accelerator for unconditional diffusion models," *ACM Trans. Embedded Comput. Syst.*, vol. 25, no. 3, pp. 1–21, May 2026.
- [22] S. Tang, P. Zheng, R. Chen, Y. Lv, B. Da Silva, and M. Ling, "Diff-DiT: Temporal differential accelerator for low-bit diffusion transformers on FPGA," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, Oct. 2025, pp. 1–9.
- [23] X. Lin et al., "FLAME: Fully leveraging MoE sparsity for transformer on FPGA," in *Proc. 61st ACM/IEEE Design Autom. Conf.*, Jun. 2024, pp. 1–6.
- [24] J. Dong et al., "UbiMoE: A ubiquitous mixture-of-experts vision transformer accelerator with hybrid computation pattern on FPGA," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2025, pp. 1–5.
- [25] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, Oct. 2012, pp. 5026–5033.
- [26] RobotStudio. (2026). *Standard Open SO-100 & SO-101 Arms*. [Online]. Available: <https://github.com/TheRobotStudio/SO-ARM100>
- [27] Y. Ma et al., "A 102.5-hz real-time FPGA robot-action-planning accelerator with velocity-estimation-driven behavior prediction for embodied artificial intelligence," in *Proc. IEEE Asian Solid-State Circuits Conf. (A-SSCC)*, Nov. 2025, pp. 76–78.
- [28] X. Liu, C. Gong, and Q. Liu, "Flow straight and fast: Learning to generate and transfer data with rectified flow," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023, pp. 1–41.
- [29] R. Cadene et al. (2024). *LeRobot: State-of-the-art Machine Learning for Real-World Robotics in Pytorch*. [Online]. Available: <https://github.com/huggingface/lerobot>
- [30] NVIDIA. (2026). *NVIDIA Ampere GA102 GPU Architecture*. [Online]. Available: <https://www.nvidia.com/content/PDF/nvidia-ampere-ga-102-gpu-architecture-whitepaper-v2.pdf>
- [31] M. Ditty, "NVIDIA ORIN system-on-chip," in *Proc. IEEE Hot Chips 34 Symp. (HCS)*, Aug. 2022, pp. 1–17.



Haozhe Zhu (Member, IEEE) received the B.E. and Ph.D. degrees in microelectronics from the School of Microelectronics, Fudan University, Shanghai, China, in 2017 and 2022, respectively.

From 2022 to 2025, he was a Post-Doctoral Research Fellow with the Frontier Institute of Chip and System, Fudan University, where he became an Assistant Professor in 2025. His current research interests include efficient integrated circuits and systems designed for intelligent applications.

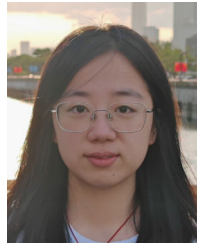
Dr. Zhu was a co-awardee of the Distinguished

Design Award from IEEE A-SSCC in 2024 and 2025.



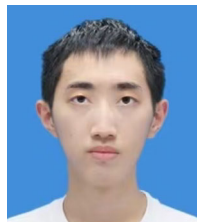
Yujie Ma (Student Member, IEEE) received the B.E. degree in microelectronics from the College of Integrated Circuits and Micro-Nano Electronics, Fudan University, Shanghai, China, in 2024, where he is currently pursuing the Ph.D. degree.

His current research interests include compute-in-memory and embodied artificial intelligence systems. He was a recipient of the Distinguished Design Award and Women-in-Circuits STGA from IEEE A-SSCC in 2025.



Siqi He (Graduate Student Member, IEEE) received the B.E. degree in microelectronics from Fudan University, Shanghai, China, in 2022, where she is currently pursuing the Ph.D. degree with the College of Integrated Circuits and Micro-Nano Electronics.

Her research interests include compute-in-memory and large language models. She was a recipient of the Distinguished Design Award and Women-in-Circuits STGA from IEEE A-SSCC in 2025.



Linqi Li (Student Member, IEEE) received the B.S. degree in microelectronics from Fudan University, Shanghai, China, in 2025, where he is currently pursuing the M.S. degree with the College of Integrated Circuits and Micro-Nano Electronics.

His research interests include digital circuit and embodied artificial intelligence system design.



Sinong Gong (Student Member, IEEE) received the B.S. degree in electronic science and technology from Northeastern University, Shenyang, China, in 2024. He is currently pursuing the Ph.D. degree with the College of Integrated Circuits and Micro-Nano Electronics, Fudan University, Shanghai, China.

His research interests include digital circuit and embodied artificial intelligence system design.



Mengjie Li (Graduate Student Member, IEEE) received the B.E. degree in electronic science and technology from Southeast University, Nanjing, China, in 2021, and the Ph.D. degree from the College of Integrated Circuits and Micro-Nano Electronics, Fudan University, Shanghai, China, in 2026.

She is currently an IC Design Engineer with the Alibaba DAMO Academy, Shanghai. Her research interests include compute-in-memory systems and domain-specific accelerators design.



Qi Liu (Member, IEEE) received the Ph.D. degree from Anhui University, Hefei, China, in 2010.

In 2016, he joined the Institute of Microelectronics (IME), Chinese Academy of Sciences (CAS), Beijing, China, as a Professor, where he is currently a Guest Professor. He is also a Professor with the Frontier Institute of Chip and System, Fudan University, Shanghai, China, and the Director of the State Key Laboratory of Integrated Chips and Systems, Fudan University. His research interests include fabrication, characterization, and mechanisms of emerging memory devices and chips for memory, logic, computing-in-memory, and neuromorphic computing.



Xiankui Xiong received the M.S. degree in electronic engineering from the University of Electronic Science and Technology of China, Chengdu, China, in 1998.

He is currently the Chief Architect of wireless with ZTE Corporation, Shenzhen, China, where he leads the Prospect Team. His research focuses on computing systems and architectures, advanced computing paradigms, and heterogeneous computing accelerators.



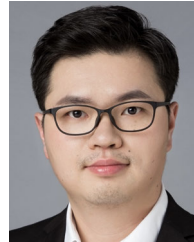
Xiaoyang Zeng (Senior Member, IEEE) received the B.S. degree from Xiangtan University, Xiangtan, China, in 1992, and the Ph.D. degree from Changchun Institute of Optics, Fine Mechanics and Physics, Chinese Academy of Sciences, Changchun, China, in 2001.

From 2001 to 2003, he was a Post-Doctoral Researcher with Fudan University, Shanghai, China. He joined the State Key Laboratory of Integrated Chips and Systems (formerly the State Key Laboratory of ASIC and System), Fudan University, as an Associate Professor, where he is currently a Full Professor and the Vice Director. He is also the Vice Director of the Institute of Science and Technology, Fudan University. His research interests include information security chip design, system-on-chip platforms, and VLSI implementation of digital signal processing and communication systems.



Chengchen Wang received the M.S. degree from the Department of Precision Instrument, Tsinghua University, Beijing, China, in 2022.

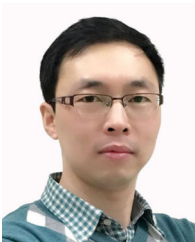
He is currently with ZTE Corporation, Shenzhen, China, focusing on advanced computing fields, such as in-memory computing, probabilistic computing, and quantum computing.



Chixiao Chen (Member, IEEE) received the B.S. and Ph.D. degrees in microelectronics from Fudan University, Shanghai, China, in 2010 and 2015, respectively.

In 2015, he was with Calterah Inc., Shanghai, as an Analog/Mixed Signal Circuit Design Engineer. From 2016 to 2018, he was a Post-Doctoral Research Associate with the University of Washington, Seattle, WA, USA. In 2019, he joined Fudan University, where he is currently an Associate Professor with the College of Integrated Circuits and Micro-Nano Electronics. He is also the Vice Director of the State Key Laboratory of Integrated Chips and Systems, Fudan University. His current research interests include mixed-signal integrated circuit design and custom intelligent software–hardware codesigns.

Dr. Chen has been served as a TPC Member for A-SSCC since 2023.



Haidong Tian received the M.S. degree in control engineering from Nanjing University of Aeronautics and Astronautics, Nanjing, China.

In 2006, he was an Electronics Engineer with ZTE Corporation, Nanjing. His current research focuses on OIO-based supernode system architecture, heterogeneous memory architecture and in-memory computing, and near-data processing.